

SKILL Week 7 OSSP (S 13 & 14)

SESSION 13:

TASK1: To parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

```
koumudi@Koumudi: ~/OSSP/ × + v
GNU nano 7.2
Hello from input redirection.
This data is coming from a file.
OSSP Session 13 Task 1.
```

```
koumudi@Koumudi: ~/OSSP/ × + v
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define INPUT_SIZE 1024

int main()
{
    char input[INPUT_SIZE];

    printf("Input Redirection Manager - Session 13 Task 1\n");
    printf("Use: command < filename\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        printf("input-shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            break;
        }

        if (strlen(input) == 0)
        {
            continue;
        }

        /*
         * Find the input redirection operator <
         */
        char *redirect = strchr(input, '<');
        if (redirect == NULL)
```

```
koumudi@Koumudi: ~/OSSP/ × + v
GNU nano 7.2

        if (redirect == NULL)
        {
            printf("Error: Input redirection '<' not found.\n");
            continue;
        }

        /*
         * Separate command and filename.
         */
        *redirect = '\0';

        char *command = input;
        char *filename = redirect + 1;

        while (*command == ' ' || *command == '\t')
        {
            command++;
        }

        while (*filename == ' ' || *filename == '\t')
        {
            filename++;
        }

        /*
         * Open the input file.
         */
        int fd = open(filename, O_RDONLY);

        if (fd == -1)
        {
            perror("open");
            printf("Error: Could not open input file '%s'.\n",
                filename);
            continue;
        }

        /*
         * Save the original stdin.
         */
        int saved_stdin = dup(STDIN_FILENO);

        if (saved_stdin == -1)
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
if (saved_stdin == -1)
{
    perror("dup");
    close(fd);
    continue;
}

/*
 * Redirect stdin to the input file.
 */
if (dup2(fd, STDIN_FILENO) == -1)
{
    perror("dup2");
    close(fd);
    close(saved_stdin);
    continue;
}

close(fd);

printf("\nInput redirection successful.\n");
printf("Reading data from: %s\n", filename);

/*
 * Read redirected input.
 */
char buffer[INPUT_SIZE];
while (fgets(buffer, sizeof(buffer), stdin) != NULL)
{
    printf("INPUT: %s", buffer);
}

/*
 * Restore original stdin.
 */
if (dup2(saved_stdin, STDIN_FILENO) == -1)
{
    perror("dup2 restore");
}

close(saved_stdin);
printf("\nInput stream restored.\n\n");
```

```
    printf("\nInput stream restored.\n\n");
}

printf("Input redirection manager terminated.\n");

return 0;
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
koumudi@Koumudi:~$ cd ~/OSSP/SKILL/Session_13
koumudi@Koumudi:~/OSSP/SKILL/Session_13$ pwd
/home/koumudi/OSSP/SKILL/Session_13
koumudi@Koumudi:~/OSSP/SKILL/Session_13$ ls
input.txt  input_redirect  input_redirect.c
koumudi@Koumudi:~/OSSP/SKILL/Session_13$ cat input.txt
Hello from input redirection.
This data is coming from a file.
OSSP Session 13 Task 1.
koumudi@Koumudi:~/OSSP/SKILL/Session_13$ ./input_redirect
Input Redirection Manager - Session 13 Task 1
Use: command < filename
Type 'exit' to quit.

input-shell> cat < input.txt

Input redirection successful.
Reading data from: input.txt

INPUT: Hello from input redirection.
INPUT: This data is coming from a file.
INPUT: OSSP Session 13 Task 1.

Input stream restored.

input-shell> Input redirection manager terminated.
koumudi@Koumudi:~/OSSP/SKILL/Session_13$ cat < missing.txt
-bash: missing.txt: No such file or directory
koumudi@Koumudi:~/OSSP/SKILL/Session_13$ cat < input.txt
Hello from input redirection.
This data is coming from a file.
OSSP Session 13 Task 1.
koumudi@Koumudi:~/OSSP/SKILL/Session_13$ |
```

TASK2: To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

```
koumudi@Koumudi: ~/OSSP/ × + v
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

#define INPUT_SIZE 1024

int main()
{
    char input[INPUT_SIZE];

    printf("Output Redirection Manager - Session 13 Task 2\n");
    printf("Use: command > filename\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        printf("output-shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            break;
        }

        if (strlen(input) == 0)
        {
            continue;
        }

        /*
         * Find output redirection operator >
         */
        char *redirect = strchr(input, '>');

        if (redirect == NULL)
        {
            printf("Error: Output redirection '>' not found.\n");
            continue;
        }

        /*
         * Separate command and filename.
         */
        *redirect = '\0';
```

```
koumudi@Koumudi: ~/OSSP/ × + v
GNU nano 7.2
*redirect = '\0';

char *command = input;
char *filename = redirect + 1;

while (*command == ' ' || *command == '\t')
{
    command++;
}

while (*filename == ' ' || *filename == '\t')
{
    filename++;
}

if (strlen(command) == 0 ||
    strlen(filename) == 0)
{
    printf("Error: Invalid output redirection.\n");
    continue;
}

/*
 * Save original stdout.
 */
int saved_stdout = dup(STDOUT_FILENO);

if (saved_stdout == -1)
{
    perror("dup");
    continue;
}

/*
 * Create/open output file.
 *
 * O_TRUNC means existing contents
 * will be replaced.
 */
int fd = open(
    filename,
    O_WRONLY | O_CREAT | O_TRUNC,
    0644
);

if (fd == -1)
{
    perror("open");
    printf(
        "Error: Could not create output file '%s'.\n",
        filename
    );
}
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
/*Error: Could not create output file '%s'.\n",
filename
);
close(saved_stdout);
continue;
}

/*
 * Redirect stdout to the file.
 */
if (dup2(fd, STDOUT_FILENO) == -1)
{
    perror("dup2");
    close(fd);
    close(saved_stdout);
    continue;
}

close(fd);

/*
 * Execute a simple command using popen.
 *
 * The command output is copied
 * into the redirected stdout.
 */
FILE *pipe_stream = popen(command, "r");
if (pipe_stream == NULL)
{
    /*
     * Restore stdout before printing error.
     */
    dup2(saved_stdout, STDOUT_FILENO);
    close(saved_stdout);
    perror("popen");
    continue;
}

char buffer[INPUT_SIZE];
while (fgets(
    buffer,
    sizeof(buffer),
    pipe_stream) != NULL)
{
    printf("%s", buffer);
}
|
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
    dup2(saved_stdout, STDOUT_FILENO);
    close(saved_stdout);
    perror("popen");
    continue;
}

char buffer[INPUT_SIZE];

while (fgets(
    buffer,
    sizeof(buffer),
    pipe_stream) != NULL)
{
    printf("%s", buffer);
}

int status = pclose(pipe_stream);

/*
 * Restore stdout.
 */
if (dup2(saved_stdout, STDOUT_FILENO) == -1)
{
    perror("dup2 restore");
}

close(saved_stdout);

if (status == -1)
{
    printf("Command execution failed.\n");
}
else
{
    printf(
        "Output redirection successful.\n"
    );
    printf(
        "Output saved to: %s\n\n",
        filename
    );
}

printf("Output redirection manager terminated.\n");

return 0;
}
```

```
koumudi@Koumudi: ~/OSSP/ × + ∨  
Hello from input redirection.  
This data is coming from a file.  
OSSP Session 13 Task 1.  
koumudi@Koumudi:~/OSSP/SKILL/Session_13$ cd ~/OSSP/SKILL/Session_13  
koumudi@Koumudi:~/OSSP/SKILL/Session_13$ pwd  
/home/koumudi/OSSP/SKILL/Session_13  
koumudi@Koumudi:~/OSSP/SKILL/Session_13$ nano output_redirect.c  
koumudi@Koumudi:~/OSSP/SKILL/Session_13$ gcc output_redirect.c -o output_redirect  
koumudi@Koumudi:~/OSSP/SKILL/Session_13$ ls  
input.txt input_redirect input_redirect.c output_redirect output_redirect.c  
koumudi@Koumudi:~/OSSP/SKILL/Session_13$ ./output_redirect  
Output Redirection Manager - Session 13 Task 2  
Use: command > filename  
Type 'exit' to quit.  
  
output-shell> echo Hello OSSP > output.txt  
Output redirection successful.  
Output saved to: output.txt  
  
output-shell> cat output.txt  
Error: Output redirection '>' not found.  
output-shell> ls > directory.txt  
Output redirection successful.  
Output saved to: directory.txt  
  
output-shell> cat directory.txt  
Error: Output redirection '>' not found.  
output-shell> echo First Data > test.txt  
Output redirection successful.  
Output saved to: test.txt  
  
output-shell> cat test.txt  
Error: Output redirection '>' not found.  
output-shell> echo Second Data > test.txt  
Output redirection successful.  
Output saved to: test.txt  
  
output-shell> cat test.txt  
Error: Output redirection '>' not found.  
output-shell> |
```

SESSION 14:

TASK1: To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

#define INPUT_SIZE 1024

int main()
{
    char input[INPUT_SIZE];

    printf("Append Redirection Manager - Session 14 Task 1\n");
    printf("Use: command >> filename\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        printf("append-shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            break;
        }

        if (strlen(input) == 0)
        {
            continue;
        }

        /*
         * Find the append operator >>
         */
        char *redirect = strstr(input, ">>");
    }

```

```

GNU nano 7.2
    if (redirect == NULL)
    {
        printf("Error: Append operator '>>' not found.\n");
        continue;
    }

    /*
     * Separate command and filename.
     */
    *redirect = '\0';

    char *command = input;
    char *filename = redirect + 2;

    while (*command == ' ' || *command == '\t')
    {
        command++;
    }

    while (*filename == ' ' || *filename == '\t')
    {
        filename++;
    }

    if (strlen(command) == 0 ||
        strlen(filename) == 0)
    {
        printf("Error: Invalid append redirection.\n");
        continue;
    }

    /*
     * Save original stdout.
     */
    int saved_stdout = dup(STDOUT_FILENO);

    if (saved_stdout == -1)
    {
        perror("dup");
        continue;
    }

```

```

GNU nano 7.2
    }
    continue;

    /*
     * Open file in append mode.
     * O_APPEND preserves existing data.
     */
    int fd = open(
        filename,
        O_WRONLY | O_CREAT | O_APPEND,
        0644
    );

    if (fd == -1)
    {
        perror("open");
        printf(
            "Error: Could not open file '%s'.\n",
            filename
        );

        close(saved_stdout);
        continue;
    }

    /*
     * Redirect stdout to the file.
     */
    if (dup2(fd, STDOUT_FILENO) == -1)
    {
        perror("dup2");

        close(fd);
        close(saved_stdout);

        continue;
    }

    close(fd);

    /*

```

```

GNU nano 7.2
    continue;
    }

    char buffer[INPUT_SIZE];

    while (fgets(
        buffer,
        sizeof(buffer),
        pipe_stream) != NULL)
    {
        printf("%s", buffer);
    }

    int status = pclose(pipe_stream);

    /*
     * Restore stdout.
     */
    if (dup2(saved_stdout, STDOUT_FILENO) == -1)
    {
        perror("dup2 restore");
    }

    close(saved_stdout);

    if (status == -1)
    {
        printf("Command execution failed.\n");
    }
    else
    {
        printf("Append redirection successful.\n");
        printf("Data appended to: %s\n\n",
            filename);
    }
}

printf("Append redirection manager terminated.\n");

return 0;
}

```

```

    return 0;
}
koumudi@Koumudi: ~/OSSP/Session_14$ gcc append_redirect.c -o append_redirect
koumudi@Koumudi: ~/OSSP/Session_14$ rm -f append.txt
koumudi@Koumudi: ~/OSSP/Session_14$ ./append_redirect
Append Redirection Manager - Session 14 Task 1
Use: command >> filename
Type 'exit' to quit.

append-shell> echo First Line >> append.txt
Append redirection successful.
Data appended to: append.txt

append-shell> echo Second Line >> append.txt
Append redirection successful.
Data appended to: append.txt

append-shell> echo Third Line >> append.txt
Append redirection successful.
Data appended to: append.txt

append-shell> exit
Append redirection manager terminated.
koumudi@Koumudi: ~/OSSP/Session_14$ cat append.txt
First Line
Second Line
Third Line
koumudi@Koumudi: ~/OSSP/Session_14$ ls -l append.txt
-rw-r--r-- 1 koumudi koumudi 34 Sep 19 02:50 append.txt
koumudi@Koumudi: ~/OSSP/Session_14$ |

```

TASK2: To parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define INPUT_SIZE 1024
#define MAX_ARGS 64

char *trim(char *str)
{
    while (*str == ' ' || *str == '\t')
    {
        str++;
    }

    if (*str == '\0')
    {
        return str;
    }

    char *end = str + strlen(str) - 1;

    while (end > str &&
           (*end == ' ' || *end == '\t'))
    {
        end--;
    }

    *(end + 1) = '\0';
    return str;
}

int tokenize_command(char *command, char **args)
{
    int count = 0;

    char *token = strtok(command, " \t");

    while (token != NULL &&

```

```

    token = strtok(NULL, " \t");
    }

    args[count] = NULL;
    return count;
}

int main()
{
    char input[INPUT_SIZE];

    printf("stderr Redirection Manager - Session 14 Task 2\n");
    printf("Use: command 2> error_file\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        printf("stderr-shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            break;
        }

        if (strlen(input) == 0)
        {
            continue;
        }

        char *redirect = strstr(input, "2>");

        if (redirect == NULL)
        {

```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
if (redirect == NULL)
{
    printf("Error: stderr redirection '2>' not found.\n");
    continue;
}

*redirect = '\0';

char *command = trim(input);
char *filename = trim(redirect + 2);

if (strlen(command) == 0 ||
    strlen(filename) == 0)
{
    printf("Error: Invalid stderr redirection.\n");
    continue;
}

char command_copy[INPUT_SIZE];

strncpy(
    command_copy,
    command,
    INPUT_SIZE - 1
);

command_copy[INPUT_SIZE - 1] = '\0';

char *args[MAX_ARGS];

if (tokenize_command(
    command_copy,
    args) == 0)
{
    printf("Error: Empty command.\n");
    continue;
}

int fd = open(
    filename,
    O_WRONLY | O_CREAT | O_TRUNC,
    0644
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
    exit(EXIT_FAILURE);
}

close(fd);

int status;

waitpid(pid, &status, 0);

if (WIFEXITED(status))
{
    if (WEXITSTATUS(status) == 0)
    {
        printf(
            "Command executed successfully.\n"
        );
    }
    else
    {
        printf(
            "Command failed.\n"
        );
    }
}
else
{
    printf(
        "Command did not terminate normally.\n"
    );
}

printf(
    "stderr output saved to: %s\n\n",
    filename
);

printf("stderr redirection manager terminated.\n");

return 0;
}
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾

stderr-shell> ls /this_folder_does_not_exist 2> errors.txt
Command failed.
stderr output saved to: errors.txt

stderr-shell> exit
stderr redirection manager terminated.
koumudi@Koumudi:~/OSSP/SKILL/Session_14$ cat errors.txt
ls: cannot access '/this_folder_does_not_exist': No such file or directory
koumudi@Koumudi:~/OSSP/SKILL/Session_14$ ./stderr_redirect
stderr Redirection Manager - Session 14 Task 2
Use: command 2> error_file
Type 'exit' to quit.

stderr-shell> echo Hello OSSP 2> errors2.txt
Hello OSSP
Command executed successfully.
stderr output saved to: errors2.txt

stderr-shell> exit
stderr redirection manager terminated.
koumudi@Koumudi:~/OSSP/SKILL/Session_14$ cat errors2.txt
cat: /file_that_does_not_exist: No such file or directory
koumudi@Koumudi:~/OSSP/SKILL/Session_14$ ./stderr_redirect
stderr Redirection Manager - Session 14 Task 2
Use: command 2> error_file
Type 'exit' to quit.

stderr-shell> cat /file_that_does_not_exist 2> cat_error.txt
Command failed.
stderr output saved to: cat_error.txt

stderr-shell> exit
stderr redirection manager terminated.
koumudi@Koumudi:~/OSSP/SKILL/Session_14$ cat cat_error.txt
cat: /file_that_does_not_exist: No such file or directory
koumudi@Koumudi:~/OSSP/SKILL/Session_14$ ./stderr_redirect
stderr Redirection Manager - Session 14 Task 2
Use: command 2> error_file
Type 'exit' to quit.

stderr-shell> ls
Error: stderr redirection '2>' not found.
stderr-shell> exit
stderr redirection manager terminated.
koumudi@Koumudi:~/OSSP/SKILL/Session_14$ ls
append.txt  append_redirect  append_redirect.c  cat_error.txt  errors.txt  errors2.txt  stderr_redirect  stderr_redirect.c
koumudi@Koumudi:~/OSSP/SKILL/Session_14$ |
```